

Map-Guided Visual Geolocation Agent: Full Development Plan

1. Project Summary

This project builds a GeoGuessr-style AI system that predicts the geographic location of a street-view image by interacting with a map. Instead of directly predicting a country or latitude/longitude in one shot, the model repeatedly looks at the street-view image and the current map view, chooses likely map tiles to zoom into, and eventually predicts an exact coordinate.

The key idea is:

```
Street-view image + current map screenshot + map bounds
↓
Model predicts probabilities over a 16×16 map grid
↓
Keep top-k tiles using beam search
↓
Zoom into candidate tiles
↓
Repeat for several steps
↓
Final click prediction → latitude/longitude
```

The final system combines:

```
computer vision
map-image understanding
geospatial projection math
supervised imitation learning
beam search
local map rendering infrastructure
interactive deployment
```

The project is designed as a strong internship portfolio project because it includes both machine learning and real engineering infrastructure.

2. Final System Goal

Given one street-view image, the system should output:

```
{
  "predicted_location": {
    "lat": 35.6806,
    "lon": 139.7642
  },
  "confidence_radius_km": 18.7,
  "search_path": [
    {"step": 1, "selected_tile": 173, "confidence": 0.31},
    {"step": 2, "selected_tile": 94, "confidence": 0.44},
    {"step": 3, "selected_tile": 122, "confidence": 0.52}
  ],
  "top_candidate_locations": [
    {"lat": 35.6806, "lon": 139.7642, "score": 0.61},
    {"lat": 37.5665, "lon": 126.9780, "score": 0.19},
    {"lat": 25.0330, "lon": 121.5654, "score": 0.11}
  ]
}
```

The system should also visualize the map zoom path so the user can watch the model narrow down the location.

3. Main Components

The full project has five major components:

1. Dataset pipeline
2. Local map-rendering backend
3. Map-guided geolocation model
4. Training and inference system
5. Interactive demo and evaluation dashboard

High-level architecture:

```
OSV-5M street-view images + GPS labels
  ↓
Dataset loader
  ↓
Map backend generates map screenshots and tile targets
  ↓
Model trains to choose correct map tiles
  ↓
Beam-search inference generates candidate locations
  ↓
```

Final click head outputs exact latitude/longitude

↓

Demo visualizes prediction path

4. Dataset Plan

4.1 Main Dataset

Use **OpenStreetView-5M (OSV-5M)** as the main dataset.

Each sample should contain:

```
image_id
street_image_path
latitude
longitude
country
region metadata if available
```

Recommended dataset scale by phase:

```
Prototype: 50k-100k images
Early model: 250k-500k images
Serious portfolio model: 1M-2.5M images
Full research version: all OSV-5M training images
```

Since the full dataset is large, start with a subset and scale only after the pipeline works.

4.2 Sampling Strategy

Do not randomly take half the image files without checking balance. Random sampling can overrepresent countries with many images and make the model learn geographic priors instead of visual cues.

Use country-balanced or geographically balanced sampling.

Example sampling rule:

```
For each country:
  if country has fewer than 2,000 images:
    keep all images
```

```
else:
    keep up to country_cap images
```

Example settings:

```
country_cap = 50,000
minimum_keep = all images under 2,000
```

This helps preserve rare countries while preventing large countries from dominating the dataset.

4.3 Dataset Table Format

Create a processed metadata table:

```
image_id,image_path,lat,lon,country,split
abc123,images/train/abc123.jpg,35.6812,139.7671,Japan,train
xyz789,images/train/xyz789.jpg,48.8566,2.3522,France,train
```

Later, add generated training targets:

```
image_id,step,map_path,target_tile,target_u,target_v,bounds_id
abc123,0,,173,,,world
abc123,1,173,94,,,
abc123,2,173_94,122,,,
abc123,final,173_94_122,,,0.63,0.41
```

5. Local Map-Rendering Backend

The map backend is a local service that takes map coordinates or a tile path and returns a screenshot of the corresponding map view.

This avoids using Google Maps screenshots or scraping public OpenStreetMap tile servers.

5.1 Backend Goal

Input:

```
{
  "path": [173, 94, 122],
```

```

    "grid_size": 16,
    "width": 512,
    "height": 512,
    "style": "default"
}

```

Output:

```

{
  "image_path": "cache/maps/16x16/173_94_122.png",
  "bounds": {
    "lat_min": 34.9,
    "lat_max": 36.2,
    "lon_min": 138.9,
    "lon_max": 140.3
  },
  "projection": "web_mercator"
}

```

5.2 Recommended Tool Stack

Use:

```

FastAPI
+ TileServer GL
+ OpenMapTiles or Planetiler-generated vector tiles
+ MapLibre GL JS
+ Playwright
+ pyproj
+ SQLite/DuckDB cache index

```

Tool roles:

Layer	Tool	Purpose
Map data	OpenStreetMap / Natural Earth	raw geographic data
Vector tiles	OpenMapTiles / Planetiler	create or provide local <code>.mbtiles</code> / <code>.pmtiles</code>
Tile server	TileServer GL / Martin	serve local vector tiles
Map renderer	MapLibre GL JS	render visual map in browser

Layer	Tool	Purpose
Screenshot capture	Playwright	take PNG screenshots
Backend API	FastAPI	provide <code>/render</code> endpoints
Projection math	pyproj / mercantile	Web Mercator and tile math
Cache	filesystem + SQLite/DuckDB	avoid duplicate rendering

5.3 Backend Architecture

```

Training code
  ↓ request map screenshot
FastAPI backend
  ↓ opens local MapLibre page
MapLibre GL JS
  ↓ loads local vector tiles
TileServer GL
  ↓ reads local MBTiles/PMTiles
OpenMapTiles / Planetiler tiles
  ↓
Playwright screenshot
  ↓
FastAPI returns PNG path + map bounds

```

5.4 Backend Endpoints

Recommended endpoints:

```

GET  /health
POST /render/bounds
POST /render/path
POST /render/batch
POST /tile/target
POST /tile/zoom

```

`/render/path`

Input:

```
{
  "path": [173, 94, 122],
  "grid_size": 16,
  "width": 512,
  "height": 512
}
```

Output:

```
{
  "image_path": "cache/maps/16x16/173_94_122.png",
  "bounds": {
    "lat_min": 34.9,
    "lat_max": 36.2,
    "lon_min": 138.9,
    "lon_max": 140.3
  }
}
```

/render/bounds

Input:

```
{
  "lat_min": 30.0,
  "lat_max": 45.0,
  "lon_min": 125.0,
  "lon_max": 150.0,
  "width": 512,
  "height": 512
}
```

Output:

```
{
  "image_path": "cache/maps/bounds_hash.png",
  "actual_bounds": {
    "lat_min": 30.0,
    "lat_max": 45.0,
    "lon_min": 125.0,
    "lon_max": 150.0
  }
}
```

```
}  
}
```

`/render/batch`

Input:

```
{  
  "paths": [  
    [173],  
    [173, 94],  
    [173, 94, 122]  
  ],  
  "grid_size": 16,  
  "width": 512,  
  "height": 512  
}
```

This avoids many small HTTP requests during training.

5.5 Map Projection

Use **Web Mercator** for the final implementation.

Do not split raw latitude evenly if the map is rendered in Web Mercator. Most web maps are not linear in latitude.

Correct approach:

```
lat/lon bounds  
  ↓  
convert to Web Mercator x/y  
  ↓  
split x/y bounds into 16×16 grid  
  ↓  
selected tile becomes new x/y bounds  
  ↓  
convert x/y bounds back to lat/lon
```

Use `pyproj` for coordinate conversion.

5.6 Caching

Caching is essential.

Cache key options:

```
style + grid_size + width + height + tile_path
```

Example cache paths:

```
cache/maps/16x16/step_0/world.png  
cache/maps/16x16/step_1/173.png  
cache/maps/16x16/step_2/173_94.png  
cache/maps/16x16/step_3/173_94_122.png
```

Use cache lookup before rendering:

```
if cached image exists:  
    return cached path  
else:  
    render screenshot  
    save image  
    return path
```

Recommended cache storage:

```
PNG files on disk  
+ SQLite or DuckDB metadata index
```

6. Map Tile Search System

The model searches the map by repeatedly choosing tiles.

6.1 Grid Action Space

At each step, split the map view into a 16×16 grid.

```
16 × 16 = 256 actions
```

The model outputs:

```
P(tile_0), P(tile_1), ..., P(tile_255)
```

Each tile corresponds to a possible zoom target.

6.2 Tile Path

A map path is the sequence of selected tiles.

Example:

```
[]          → world map  
[173]       → zoomed into tile 173  
[173, 94]   → zoomed again  
[173, 94, 122] → further zoomed
```

This path can be used as a stable ID for caching and training.

6.3 Target Tile Generation

For each image with true GPS coordinate, generate the correct tile at every step.

Given:

```
current map bounds  
true latitude/longitude
```

Compute the tile that contains the true point.

Prototype formula:

```
u = (true_lon - lon_min) / (lon_max - lon_min)  
v = (lat_max - true_lat) / (lat_max - lat_min)  
  
tile_x = floor(16 * u)  
tile_y = floor(16 * v)  
target_tile = tile_y * 16 + tile_x
```

Final version should use Web Mercator coordinates instead of raw lat/lon interpolation.

7. Model Architecture

7.1 High-Level Model

```
Street-view image
├─ Vision encoder: DINOv2 / CLIP / ViT / ConvNeXt
└─ Optional FFT branch
    ↓
street_embedding

Current map screenshot
└─ Map encoder: CNN / ViT
    ↓
map_embedding

Map metadata
├─ bounds
├─ zoom step
└─ previous tile path
    ↓
state_embedding

street_embedding + map_embedding + state_embedding
    ↓
fusion network
    ↓
policy head: 256 tile logits
    ↓
final click head: u, v coordinate inside final map view
```

7.2 Street Image Encoder

Recommended starting encoder:

DINOv2 or CLIP

Start with the encoder frozen.

Possible options:

Encoder	Reason
DINOv2	strong scene and visual structure embeddings
CLIP	good semantic understanding of objects and text-like visual cues
ConvNeXt	strong CNN baseline
ViT	transformer baseline

Do not train the visual encoder from scratch in the first version.

7.3 Optional FFT Branch

The FFT branch captures texture and frequency-domain cues from the street image.

```

street image
  ↓
2D FFT magnitude spectrum
  ├── 1D frequency path
  └── 2D spectrum CNN path
  ↓
frequency_embedding

```

Then:

```

visual_embedding + frequency_embedding → street_embedding

```

This should be added after the baseline model works.

7.4 Map Encoder

The map encoder processes the rendered map screenshot.

```

map screenshot → CNN/ViT → map_embedding

```

It should learn map cues such as:

```

coastlines
borders
country shapes
city labels

```

```
road density
terrain
water bodies
relative geography
```

7.5 State Encoder

The state encoder processes numeric map state:

```
lat_min, lat_max, lon_min, lon_max
zoom_step
previous tile path
```

Use an MLP to produce:

```
state_embedding
```

7.6 Fusion Network

Combine all embeddings:

```
concat(street_embedding, map_embedding, state_embedding)
  ↓
LayerNorm
  ↓
MLP or small transformer block
  ↓
fused_embedding
```

Heads:

```
policy_head(fused_embedding) → 256 tile logits
click_head(fused_embedding) → final u, v
```

8. Training Algorithm

The training algorithm is **supervised imitation learning** from GPS labels.

No reinforcement learning is needed for the first version.

8.1 Training Loop Concept

For every street-view image:

1. Start with the world map.
2. Render the current map screenshot.
3. Compute which 16×16 tile contains the true GPS location.
4. Train the model to predict that tile.
5. Use teacher forcing: zoom into the true tile.
6. Repeat for T steps.
7. At the final step, train a click head to predict exact u/v inside the final map view.

8.2 Teacher Forcing

During training:

```
correct tile → next map view
```

During inference:

```
predicted tile → next map view
```

Teacher forcing makes training stable because the model always sees the correct map path during early training.

8.3 Loss Function

At each zoom step:

```
tile_loss_t = CrossEntropy(tile_logits_t, target_tile_t)
```

Final click loss:

```
click_loss = SmoothL1(pred_u, target_u) + SmoothL1(pred_v, target_v)
```

Total loss:

```
total_loss = sum(tile_loss_t for t in 0..T-1) +  $\lambda$  * click_loss
```

Recommended starting value:

```
 $\lambda$  = 1.0
```

Optional weighted version:

```
total_loss =  
    0.5 * tile_loss_0  
+ 0.7 * tile_loss_1  
+ 1.0 * tile_loss_2  
+ 1.2 * tile_loss_3  
+ 1.5 * tile_loss_4  
+ 1.0 * click_loss
```

8.4 Soft Tile Labels

Hard labels can be too strict near tile boundaries.

Instead of:

```
target tile = 73 only
```

use soft labels:

```
tile_73 = 0.60  
tile_72 = 0.15  
tile_89 = 0.15  
tile_88 = 0.10
```

Weights can be based on distance from the true point to nearby tile centers:

```
weight_i  $\propto$  exp(-distance_i2 / temperature)
```

Add soft labels after the hard-label baseline works.

8.5 Scheduled Sampling

Teacher forcing can create exposure bias because inference uses predicted tiles.

After the model works, add scheduled sampling:

```
80% use true tile
20% use model-predicted tile
```

Later:

```
60% true tile
40% predicted tile
```

This helps the model recover from imperfect paths.

9. Inference Algorithm

Use beam search instead of greedy search.

9.1 Why Beam Search

If the model picks the wrong tile early, a greedy system may never recover.

Beam search keeps multiple possible paths.

Example settings:

```
beam_k = 16
top_m = 16
num_steps = 4 or 5
grid_size = 16
```

9.2 Beam Search Process

```
Initialize beam:
  path = []
  bounds = world_bounds
```

```
score = 0

For each step:
  candidates = []

  For each path in beam:
    render map screenshot
    model predicts 256 tile probabilities
    keep top_m tile predictions

    For each selected tile:
      new_path = old_path + [tile]
      new_bounds = zoom_into_tile(old_bounds, tile)
      new_score = old_score + log(tile_probability)
      add to candidates

  beam = top beam_k candidates

After final step:
  predict final u/v click for each beam path
  convert to lat/lon
  return best or weighted top-k predictions
```

9.3 Final Coordinate Prediction

At the final map view, the model predicts:

```
u, v ∈ [0, 1]
```

Where:

```
u = horizontal click position
v = vertical click position
```

Convert to lat/lon using the final map bounds.

Prototype conversion:

```
pred_lon = lon_min + u * (lon_max - lon_min)
pred_lat = lat_max - v * (lat_max - lat_min)
```

Final implementation should use Web Mercator conversion.

10. Development Roadmap

Phase 1: Dataset Setup

Tasks:

1. Download OSV-5M metadata.
2. Select a balanced subset.
3. Download selected images or image shards.
4. Build metadata table with image path, latitude, longitude, country, split.
5. Build PyTorch dataset loader.

Deliverable:

Dataset loader returns street image + true GPS coordinate.

Phase 2: Map Math and Tile Path Generator

Tasks:

1. Define world bounds.
2. Implement Web Mercator conversion.
3. Implement `split_bounds(bounds, grid_size=16)`.
4. Implement `zoom_into_tile(bounds, tile_id)`.
5. Implement `path_to_bounds(path)`.
6. Implement `target_tile(bounds, true_latlon)`.
7. Implement `final_uv(bounds, true_latlon)`.

Deliverable:

Given a GPS coordinate, generate the correct tile path for T zoom steps.

Phase 3: Local Map Backend

Tasks:

1. Set up local vector tiles using OpenMapTiles or Planetiler.
2. Run TileServer GL or Martin locally.
3. Build MapLibre HTML renderer.
4. Use Playwright to capture screenshots.
5. Wrap screenshot rendering with FastAPI.
6. Add cache by tile path.
7. Add batch rendering endpoint.

Deliverable:

Local API returns map screenshot + bounds for any tile path.

Phase 4: Simple Model Without Map Pixels

Train the first baseline without map screenshots.

Inputs:

street image embedding
map bounds
zoom step
previous path

Tasks:

1. Precompute DINOv2/CLIP embeddings for street images.
2. Build state encoder for bounds and zoom step.
3. Train policy head to predict target tile.
4. Measure tile accuracy per step.

Deliverable:

Basic zoom policy works using street-image embedding and map state.

Phase 5: Add Map Screenshot Encoder

Tasks:

1. Render map screenshots for training paths.
2. Add map CNN/ViT encoder.
3. Fuse street embedding + map embedding + state embedding.
4. Train 8×8 version first if needed.
5. Train 16×16 version after debugging.

Deliverable:

Model uses map pixels to improve tile prediction and final geolocation.

Phase 6: Add Beam Search Inference

Tasks:

1. Implement beam state object.
2. Implement top_m expansion per beam.
3. Implement cumulative log probability scoring.
4. Render maps for candidate paths.
5. Return top-k candidate locations.
6. Visualize beam paths.

Deliverable:

Inference returns multiple candidate map paths and final locations.

Phase 7: Add Final Click Head

Tasks:

1. Compute target u/v inside final map view.
2. Add click regression head.
3. Train with SmoothL1 loss.
4. Convert predicted u/v to lat/lon.
5. Evaluate distance error.

Deliverable:

Model outputs exact latitude/longitude estimates.

Phase 8: Add FFT Branch

Tasks:

1. Compute 2D FFT magnitude spectrum from street images.
2. Build 1D frequency path.
3. Build 2D spectrum CNN path.
4. Fuse FFT embedding with visual embedding.
5. Compare with and without FFT.

Deliverable:

Frequency-augmented street-image encoder with ablation results.

Phase 9: Robust Training Improvements

Tasks:

1. Add soft tile labels.
2. Add scheduled sampling.
3. Add balanced geographic batching.
4. Add street-image augmentations.
5. Add map-style ablations if possible.

Deliverable:

More robust model that handles border cases and imperfect paths.

Phase 10: Demo Application

Features:

1. Upload a street-view image.
2. Show the map at each zoom step.

3. Display selected tiles and beam candidates.
4. Show final predicted coordinate.
5. Show top-k candidate locations.
6. Show confidence radius.
7. Show distance error if ground truth is available.

Recommended stack:

Backend: FastAPI
Frontend: Streamlit, Gradio, or React
Map display: Leaflet or MapLibre
Model: PyTorch
Experiment tracking: Weights & Biases or MLflow
Metadata: DuckDB or SQLite

Deliverable:

Interactive portfolio demo showing the AI zooming into the map.

11. Evaluation Plan

Use geolocation-specific metrics.

Main metrics:

median distance error in km
mean distance error in km
accuracy within 1 km
accuracy within 25 km
accuracy within 100 km
accuracy within 750 km
country accuracy
top-k candidate accuracy
tile accuracy by zoom step
beam search recall

Important evaluation views:

per-country error
per-continent error
urban vs rural error

high-data vs low-data country error
step-by-step tile accuracy
performance with/without map pixels
performance with/without FFT

12. Ablation Experiments

Run these comparisons:

Experiment	Purpose
Street image only	baseline geolocation capability
Street image + map bounds	tests coordinate-state usefulness
Street image + map screenshot	tests visual map reasoning
8×8 grid	easier action-space baseline
16×16 grid	precise action-space model
Greedy inference	baseline search
Beam search	tests recovery from early uncertainty
Without FFT	visual-only baseline
With FFT	tests frequency-domain contribution
Hard labels	baseline training target
Soft labels	tests border handling
Teacher forcing only	baseline training method
Scheduled sampling	tests robustness to wrong paths

The final report should honestly show which additions help and which do not.

13. Compute Plan

Do not start with expensive hardware.

Recommended compute progression:

1. Local CPU/GPU for dataset and map backend development.
2. Colab/Kaggle for small model experiments.

3. RTX 4090 rental for embedding extraction and serious training.
4. A100 only if memory or throughput becomes a real bottleneck.

Recommended rental GPU:

RTX 4090 24GB

For this project, A100/H100 is usually unnecessary unless training at very large scale.

14. Storage Plan

Expected storage components:

```
street images subset
OSV metadata
cached map screenshots
precomputed street embeddings
training target tables
model checkpoints
logs and evaluation outputs
```

Recommended organization:

```
project/
  data/
    osv5m_meta/
    images/
    processed/
  map_backend/
    tiles/
    styles/
    cache/
  models/
    checkpoints/
    embeddings/
  experiments/
    logs/
    metrics/
  app/
    backend/
    frontend/
```

Use DuckDB or Parquet for metadata and generated training targets.

15. Risks and Solutions

Risk 1: Model learns dataset priors instead of visual cues

Solution:

Use geographic balancing, per-country metrics, and hard test splits.

Risk 2: 16×16 action space is too hard early

Solution:

Start with 8×8, then upgrade to 16×16.

Risk 3: Early wrong tile ruins inference

Solution:

Use beam search instead of greedy zooming.

Risk 4: Map screenshots are expensive to generate

Solution:

Cache screenshots by tile path and pre-render common paths.

Risk 5: Map renderer and target math do not align

Solution:

Use Web Mercator consistently for rendering, splitting, target generation, and final coordinate conversion.

Risk 6: Map labels create shortcut learning

Solution:

Evaluate with label-free map styles or different map styles as an ablation.

Risk 7: Storage gets too large

Solution:

Start with 50k-100k images, cache only needed map paths, and store embeddings instead of recomputing image encodings.

16. Recommended Build Order

Build the system in this order:

1. Download OSV-5M metadata.
2. Create a small balanced image subset.
3. Build the image dataset loader.
4. Implement Web Mercator tile math.
5. Generate correct tile paths from GPS coordinates.
6. Build local map rendering backend.
7. Cache map screenshots by tile path.
8. Precompute DINOv2/CLIP street-image embeddings.
9. Train simple tile-policy model without map pixels.
10. Add map screenshot encoder.
11. Train 8×8 version.
12. Upgrade to 16×16 version.
13. Add beam search inference.
14. Add final click regression head.
15. Add FFT branch and run ablations.
16. Add soft labels and scheduled sampling.
17. Evaluate with geolocation metrics.
18. Build interactive demo.
19. Write project report and resume bullets.

17. Suggested Milestones

Milestone 1: Dataset Works

Can load image + GPS coordinate from OSV-5M subset.

Milestone 2: Tile Math Works

Can generate the correct 16×16 tile path for any GPS coordinate.

Milestone 3: Map Backend Works

Can request a path like [173, 94, 122] and receive a local map screenshot.

Milestone 4: Baseline Model Works

Frozen street-image embeddings predict correct tile sequence better than random.

Milestone 5: Map Pixels Improve Model

Adding map screenshots improves tile accuracy or distance error.

Milestone 6: Beam Search Works

Top-k search improves final geolocation accuracy over greedy search.

Milestone 7: Exact Coordinate Output Works

Final click head outputs lat/lon and distance error can be measured.

Milestone 8: Full Demo Works

User uploads an image and sees the AI zoom into the predicted location.

18. Possible Project Names

Best title:

Map-Guided Visual Geolocation Agent

Other options:

Iterative Map-Zoom Visual Geolocation
GeoZoomNet
Map-Aware GeoGuessing AI
Beam-Search Visual Geolocation Agent
Map-Guided GeoVision

19. Resume Bullets

Short version:

Built a map-guided visual geolocation agent that predicts latitude and longitude from street-view images by iteratively selecting 16×16 map tiles, performing beam-search zooming, and refining final coordinates with a click-regression head.

Detailed version:

Developed a GeoGuessr-style visual geolocation system using PyTorch, pretrained vision encoders, local OpenStreetMap rendering infrastructure, supervised tile-selection training, and beam-search inference to predict exact coordinates from street-level images.

Infrastructure-focused version:

Built a local map-rendering backend using FastAPI, MapLibre, TileServer GL, Playwright, and Web Mercator tile math to generate cached map screenshots for training a map-guided geolocation model.

ML-focused version:

Trained a supervised map-navigation model that learns to choose map zoom tiles from street-view images and map screenshots, using cross-entropy tile loss, final click regression, and top-k beam-search inference.

20. Final Summary

This project builds an AI agent that plays a GeoGuessr-like search process. It sees a street-view image and a map screenshot, predicts which tile of the map to zoom into, repeats this process with beam search, and finally predicts an exact latitude/longitude.

The project has two major strengths:

1. The model is more human-like than a direct coordinate predictor because it visually interacts with the map.
2. The engineering infrastructure is substantial because it requires a local map rendering backend, tile-path generation, caching, training data generation, beam search, and interactive visualization.

A strong first version should use:

- OSV-5M subset
- DINOv2 or CLIP street-image embeddings
- local MapLibre/TileServer GL map backend
- 8×8 or 16×16 tile policy
- teacher-forced supervised training
- beam-search inference
- final click regression
- interactive demo

Once the baseline works, add:

- FFT street-image branch
- soft tile labels
- scheduled sampling
- larger OSV-5M subset
- map-style ablations
- retrieval-based comparison

This is a technically strong, original, and portfolio-ready project plan.